Exercise 14

2D Representations and Command-Line Arguments

In this exercise, you get to write a few simple functions in x86-64 assembly language. You can download the starter files from the course webpage or you can use the following shell command to copy them directly to your work directory, if you're on remote.eos.ncsu.edu:

```
cp /mnt/coe/workspace/csc/CSC230/ex/exercise22/* .
```

You will be adding code for four functions to the subroutines.s source file. Each of the C test programs will use one of your functions, calling it one or more times and seeing if it does the right thing. You can write your functions one at a time, then compile your source file with the matching test program to see if it looks like your function does the right thing.

This is how we'll test your code when you turn it in. We may use different test programs, but we'll link your assembly language code with a separate test program to test each function. That way, even if you don't get all the functions working, you should still be able to compile successfully and link with the test programs for the functions you do complete.

The test programs use assert() to try to make sure your assembly language function does what it's supposed to. They also print out results from your function, so you can see what's going on the console output.

In your functions, you can use any CPU registers except rbx, rsp, rbp, r12, r13, r14 and r15. We haven't had a chance to talk about it yet, but these functions are considered callee-saved registers. A subroutine can't modify them unless it saves a copy and restores the register to its original value before returning. We'll learn how to do that soon; until then, we'll just avoid using these.

The following sections describe what each of your functions should do.

test-01 : Modify a global variable

For test-01.c, you need to write an assembly language function named setGlobal. This function should modify the C global variable, globalVar, by changing its value to 47. Notice that globalVar is an int, so it's four bytes in size. You need to add an instruction to copy an immediate value to this variable. A skeleton of the function is already there for you, along with an assembler directive marking setGlobal visible to code outside this component.

When your function is working, you should be able to compile and run it as follows. One of the assert statements should fail if you make a mistake. In the output below, there are no assert failures, so it looks like the function is working correctly.

```
$ gcc -Wall -std=c99 -g test-01.c subroutines.s -o test-01
$ ./test-01
before subroutine: -1
after subroutine: 47
```

If you're building on a newer installation of gcc, you may need to give the **-no-pie** option:

```
$ gcc -Wall -std=c99 -g -no-pie test-01.c subroutines.s -o test-01
$ ./test-01
before subroutine: -1
after subroutine: 47
```

test-02 : Swap the values of two global variables

For test-02.c, you need to write an assembly language function named swapGlobals. This function should exchange the values of two global variables, globalA and globalB. So, after the function executes, globalA should contain the value that was previously in globalB, and globalB should contain what was previously in globalA. Inside your function, you can use a CPU register as temporary storage; you don't really need a separate integer in memory for this. Also, remember that you can't read and write memory in the same instruction. You have to do that in two steps.

The starter code just has a label for this function. You'll need to add the rest of the code yourself, including the ret instruction at the end and the directive to expose your function to the linker. You should be able to build and test your solution with the following. If you're using a newer system, you may need the **-no-pie** compiler option.

```
$ gcc -Wall -std=c99 -g test-02.c subroutines.s -o test-02
$ ./test-02
Before variable swap: 99 -45
After variable swap: -45 99
```

test-03 : Return a result based on six function parameters

For test-03.c, you need to write an assembly language function named sixParams. This function takes six integer parameters, a, b, c, d, e and f (there's a prototype in the test-03.c source file). Your assembly language function should compute the signed integer value of **a * d + b * e - c * f** and return it as the function's return value. You'll need to add a label and a body for this function, and you'll need to tell the assembler to expose this function for the linker. You should write your code so it follows the usual precedence for arithmetic operators, doing multiply operations before addition or subtraction.

The test program calls your function a few times with different parameter values, reporting the value it returns and checking it against an expected result. You can build and run it as follows:

```
$ gcc -Wall -std=c99 -g test-03.c subroutines.s -o test-03
$ ./test-03
Result 1: -100
Result 2: 103
Result 3: 1806833073
```

test-04 : Function using Parameters, Immediate values and Globals

For test-04.c, you need to write an assembly language function named formula. This function takes two int parameters, x and y. It uses these parameters and two global int variables, globalA and globalB to compute the result of **x * (100 - globalA * y) + 3 * globalB * globalB**. Organize your code so it computes this result using the regular operator precedence for addition, subtraction and multiplication. The starter doesn't have any code for this function. You'll need to add everything yourself.

The test program calls your function a few times, with different values for the global variables and the parameters. You can build and run it as follows:

```
$ gcc -Wall -std=c99 -g test-04.c subroutines.s -o test-04
$ ./test-04
Result 1: 532
Result 2: 13970
Result 3: -122973
```

Submitting your Solution

Even if you developed your **subroutines.s** implementation one function at a time, your final solution should be

able to pass all the tests from all four test programs. You can go back and rebuild the older tests to make sure everything is still working. Each of the test programs just exercises one of the assembly language functions.

When you're done, submit the source file, subroutines.s, to the exercise_22 assignment in Moodle.